PRESENTATION SPEECH

Traffic Sign and Signal Recognition in Bad Conditions

Keshav Kumar · 20241CAI0072 · PSAIAC

Running order. 8 slides, about **15 min** of speaking at a calm pace (1,936 words), plus time for questions.

Slide	Title	What it says	Time
1	Traffic Sign and Signal Recognition in Bad Conditions	Who I am, the project, and four headline results	1 min 15 s
2	Signs must be read in the conditions where vision fails	Damage hurts recognition; the five objectives	1 min 30 s
3	Solution: measure, enhance with fuzzy logic, then classify	One photo through eight steps	1 min 45 s
4	Architecture: four layers, from datasets to the app	Four layers and the tools	1 min 30 s
5	7 datasets, 51 classifier runs, every test set scored once	Seven datasets and their purpose, total size, seeds, and an honest test protocol	2 min 30 s
6	Three courses, one pipeline	What each course contributes	2 min 45 s
7	Results: training on damage beats repairing it	The numbers behind the main lesson, and the detector scores	2 min
8	A working app, and what we learned	The app, conclusions and next steps	1 min 30 s

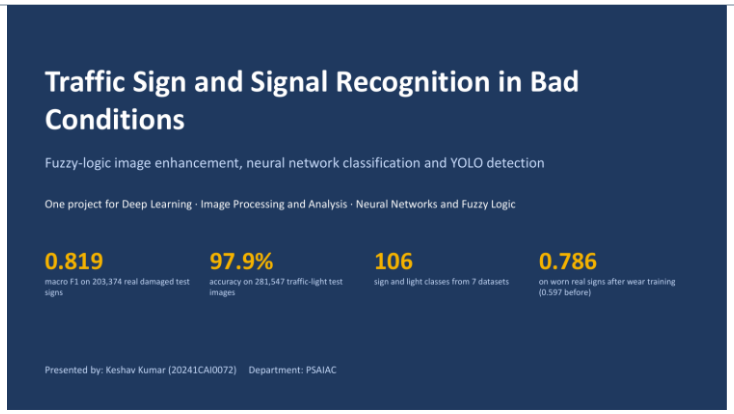
How to use this script

- Words in grey brackets are actions, such as where to point. Do not read them aloud.
- Pause for a moment after each **bold** number, and point to it on the slide.
- Rehearse twice with a timer. If you are short of time, shorten slides 4 and 5, and keep slide 7 in full: it carries the main result.
- Speak slowly. The times assume a calm pace of about 130 words a minute.

SLIDE 1 OF 8 · ABOUT 1 MIN 15 S

Traffic Sign and Signal Recognition in Bad Conditions

Key message: Who I am, the project, and four headline results



Good morning, respected faculty members and dear friends. I am **Keshav Kumar**, roll number **20241CAI0072**, from PSAIAC. Today I will present my mini project: **Traffic Sign and Signal Recognition in Bad Conditions**.

This single project covers three courses: Deep Learning, Image Processing and Analysis, and Neural Networks and Fuzzy Logic. In simple words, the system reads traffic signs and traffic lights from damaged images: photos taken at night, in haze or rain, or of old signs with worn paint. Fuzzy logic decides how to repair each image, and a neural network names the sign.

[Point to the four numbers.] Here are our four main results. Our best model scores **0.819 macro F1** on **203,374** real damaged test signs. Macro F1 is the average F1 score over all classes, so rare signs count as much as common ones. It is **97.9%** accurate on traffic lights. The final model knows **106 classes** from **7 datasets**. And on worn real signs, it improved from **0.597 to 0.786** after we trained it on worn paint.

SLIDE 2 OF 8 · ABOUT 1 MIN 30 S

Signs must be read in the conditions where vision fails

Key message: Damage hurts recognition; the five objectives



Let me start with the problem. A driver-assistance camera does not always see a clean sign. Night, haze, rain, snow, noise, blur and worn paint all damage the image. *[Point to the sample grid.]* These are signs from the CURE-TSR dataset: first clean, then darkened, noisy and hazy, at damage levels 3 and 5.

This damage really hurts a neural network. A CNN trained only on clean images reaches just **0.683** macro F1 on real damaged signs. The same CNN trained on damaged images reaches **0.819**. So our question was: can classical image processing, guided by fuzzy logic, repair the image enough to help? And how does that compare with simply training on damaged images?

[Point to the five objectives.] To answer it, we set five objectives. **One**, image processing: filters, histograms and convolution, measured with PSNR and SSIM. **Two**, fuzzy logic: a Mamdani controller that decides how strongly to denoise, stretch and darken each image. **Three**, neural networks: an MLP, an MLP on HOG and colour features, and backpropagation written by hand. **Four**, deep learning: CNNs on clean, damaged and enhanced images, growing from 17 to 106 classes, including worn signs. And **five**: a YOLO detector and a Streamlit app that show every step, with honest scoring.

SLIDE 3 OF 8 · ABOUT 1 MIN 45 S

Solution: measure, enhance with fuzzy logic, then classify

Key message: One photo through eight steps



This slide shows how one photo moves through the system. The colours show which course each step belongs to. *[Follow the arrows with your hand.]*

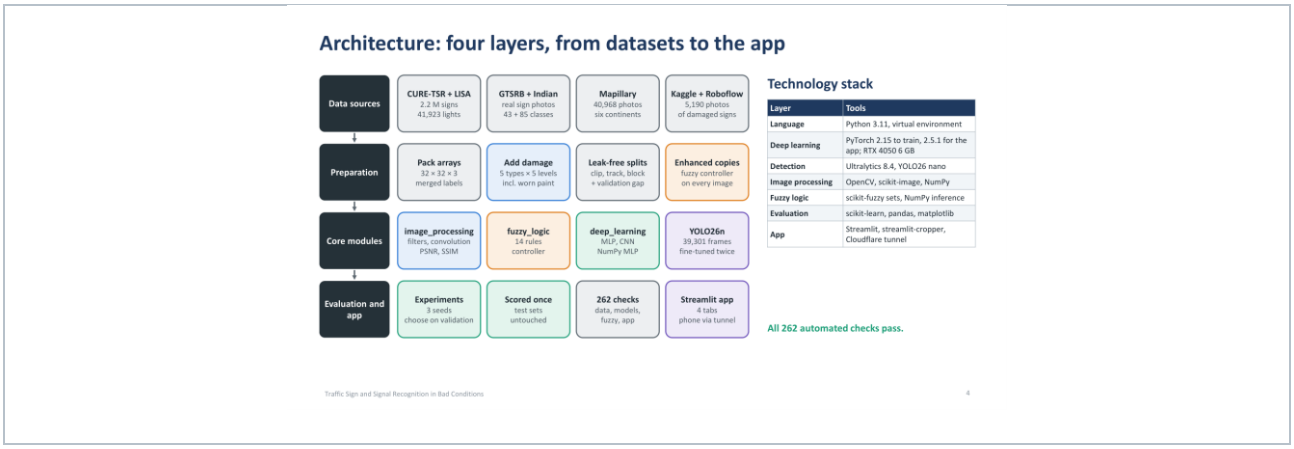
Step 1, the input: a phone or street photo, or a damaged sample. **Step 2**: a YOLO26 nano detector finds the signs and lights, and keeps every box with a confidence of 0.30 or more. If it misses a sign, the user can draw a box by hand. **Step 3**: each box is cut out as a square with a small margin, and resized to 32 by 32 colour pixels.

Step 4: we measure the quality of the crop: brightness as the mean, contrast as the RMS, and noise as colour roughness. **Step 5**: these three numbers go into a fuzzy controller with **14 Mamdani rules**. Using centroid defuzzification, it gives three strengths between 0 and 1. **Step 6**: the image is enhanced in a fixed order: Gaussian denoising, then an RGB levels stretch, then gamma darkening. Denoising comes first, because stretching would make any leftover noise stronger.

Step 7: the CNN classifies both the original crop and the enhanced crop, over 106 classes. **Step 8**: the app shows both answers side by side, with the top 3 guesses, the confidence, the rules that fired, and PSNR and SSIM. So we can always see whether enhancement changed the answer.

Architecture: four layers, from datasets to the app

Key message: Four layers and the tools



The architecture has four layers, from the datasets to the app. *[Point to each column, left to right.]*

First, the **data sources**: CURE-TSR with 2.2 million sign images, LISA with 41,923 traffic lights, GTSRB and an Indian set with real sign photos, Mapillary with 40,968 street photos from six continents, and Kaggle and Roboflow with 5,190 photos of damaged signs.

Second, **preparation**. All images are packed once into 32 by 32 arrays with merged labels. This is what made training on 1.9 million images possible on a laptop GPU. We add synthetic damage, five types at five levels, including worn paint, split the data without leakage, and make an enhanced copy of every image with the fuzzy controller.

Third, the **core modules**: image processing, fuzzy logic and deep learning, plus the YOLO26 nano detector, trained on 39,301 frames and fine-tuned twice. Fourth, **evaluation and the app**: every experiment uses 3 seeds, is chosen on validation data, and is scored once on untouched test sets. **262 automated checks** test the data, models, fuzzy logic and app, and all of them pass.

[Point to the table.] The technology stack is Python 3.11 and PyTorch on an RTX 4050 laptop GPU, Ultralytics for YOLO, OpenCV and scikit-image for image processing, scikit-fuzzy for fuzzy sets, scikit-learn for evaluation, and Streamlit for the app.

SLIDE 5 OF 8 · ABOUT 2 MIN 30 S

7 datasets, 51 classifier runs, every test set scored once

Key message: Seven datasets and their purpose, total size, seeds, and an honest test protocol



We used **seven datasets**, about **2.36 million images** in total. That is 108 GB downloaded, of which about **63 GB** was used. *[Point to the table.]* CURE-TSR is the core: 14 signs under 12 kinds of damage at 5 levels, over 2.2 million images, about 94% of the total. Its big advantage is that every damaged image has a clean original, so we can measure exactly how much the fuzzy repair helps, with PSNR and SSIM.

Each of the other datasets has a clear purpose. **LISA** gives red, yellow and green lights by day and night. With the 14 CURE-TSR signs, this makes our first **17 classes**, and its full frames teach YOLO to find lights. **GTSRB** and the **Indian set** add real German and Indian sign photos, which took us to **84 classes**. **Mapillary** adds street signs from six continents and an 'other sign' class, which took us to **106 classes**, and its full street photos teach YOLO to find signs. Finally, **Kaggle** and **Roboflow** give real damaged signs: faded, rusted and vandalised. We used them to train the CNN on worn signs and to fine-tune YOLO.

[Point to the four numbers.] In total we did **51 classifier training runs**: 17 model setups, each trained with 3 seeds. We also trained **2 YOLO detectors** in 9.4 hours, and the final CNN learned from **1.92 million** training images.

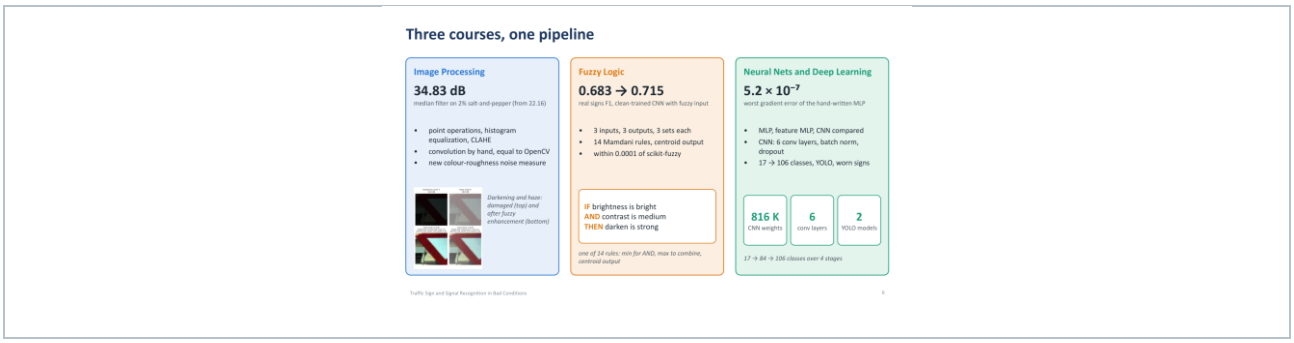
What does '3 seeds' mean? A seed is the starting number for the computer's random generator. Training is partly random: the starting weights, the order of the images, and dropout. So the same model trained twice gives slightly different scores. We train every model three times and report the average and the spread, so no result depends on one lucky run.

We were also careful to avoid data leakage. Splits are made by whole clip, track or block of frames, near-duplicates are removed, and the test street photos never enter training. *[Point to the bottom row.]* Every model follows the same protocol: train with three seeds, choose on validation data, and score the test set only once. We never changed a model after seeing its test score.

SLIDE 6 OF 8 · ABOUT 2 MIN 45 S

Three courses, one pipeline

Key message: What each course contributes



This slide shows what each course contributes. *[Point to each column.]*

Image Processing. We implemented point operations, histogram equalization and CLAHE, which is contrast-limited adaptive histogram equalization. We wrote convolution by hand, and it gives the same result as OpenCV. We also designed a new noise measure based on colour roughness. For example, a median filter on 2% salt-and-pepper noise raised PSNR from 22.16 to **34.83 decibels**.

Fuzzy Logic. The fuzzy controller decides how much to repair each image. It has 3 inputs: brightness, which can be dark, normal or bright; contrast, which can be low, medium or high; and noise, which can be low, medium or high. It has 3 outputs, denoise, stretch and darken, each none, weak or strong. There are 14 Mamdani rules. For example: if brightness is dark, stretch is strong, which fixes night images. If contrast is low, stretch is strong, which fixes haze. If noise is high, denoise is strong. And if noise is low, denoise is none, so clean images are not blurred. *[Point to the rule box.]* The rule on the slide fixes over-exposed images: IF brightness is bright AND contrast is medium, THEN darken is strong. We use min for AND, max to combine rules, and the centroid for the output, and our own inference matches scikit-fuzzy within 0.0001.

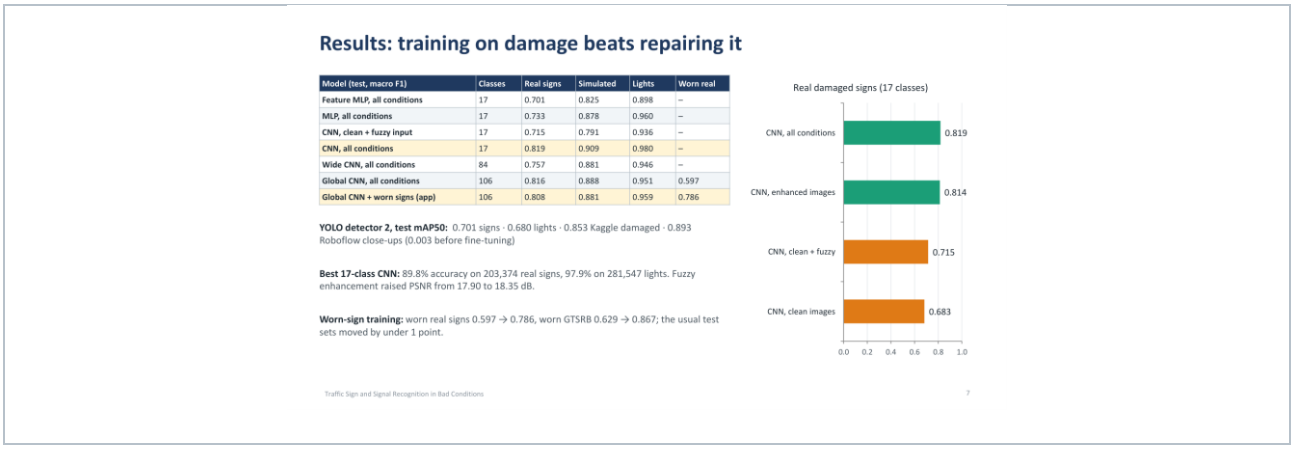
Why fuzzy logic and not plain if-else? Because 'dark' has no sharp line. With if-else, two images that differ only slightly in brightness could get completely different repairs. Fuzzy logic lets an image be partly dark and partly normal, so the repair changes smoothly, and a clean image is left almost unchanged. With fuzzy input, the clean-trained CNN rose from **0.683 to 0.715** on real signs. *[Point to the images.]* These are darkened and hazy signs before and after fuzzy enhancement.

Neural Networks and Deep Learning. We compared an MLP, a feature MLP and a CNN. In the MLP, backpropagation was written by hand, and its worst gradient error was only **5.2 times 10 to the power minus 7**. The CNN has six convolution layers with batch normalization and dropout, and about **816 thousand** weights. The hand-written MLP shows that we understand backpropagation, not just use it.

SLIDE 7 OF 8 · ABOUT 2 MIN

Results: training on damage beats repairing it

Key message: The numbers behind the main lesson, and the detector scores



Now the results. All scores are test-set macro F1, the average of three seeds, measured once. The three seeds differ only a little: for example, our best CNN scores 0.819 plus or minus 0.003. So a gap like 0.683 against 0.819 is far bigger than chance.

The main lesson is in the title: **training on damage beats repairing it.** *[Point to the chart.]* A CNN trained on clean images scores 0.683 on real damaged signs. Fuzzy enhancement lifts it to 0.715. But a CNN trained on enhanced images reaches 0.814, and one trained on all conditions reaches **0.819**. So enhancement helps a model that has never seen damage, but training on damaged images helps much more.

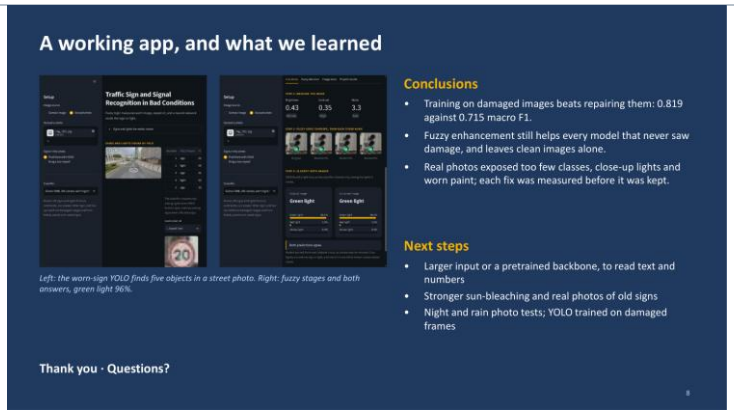
[Point to the table.] The 17-class CNN trained on all conditions is the best: 0.819 on real signs, 0.909 on simulated signs and 0.980 on lights. That is **89.8%** accuracy on 203,374 real signs and **97.9%** on 281,547 light images. The CNN clearly beats both MLPs. The 106-class CNN keeps almost the same quality, 0.816 on real signs, while knowing six times more classes. After worn-sign training, worn real signs went from **0.597 to 0.786**, and worn GTSRB signs from 0.629 to 0.867, while the usual test sets moved by less than one point. This is the model the app uses.

[Point to the line under the table.] For detection, our second YOLO detector reaches an mAP50, the mean average precision at 50% box overlap, of 0.701 on signs, 0.680 on lights and 0.853 on Kaggle damaged signs. On Roboflow close-ups it went from 0.003 before fine-tuning to **0.893**. Finally, fuzzy enhancement raised image quality, measured as PSNR, from 17.90 to 18.35 dB.

SLIDE 8 OF 8 · ABOUT 1 MIN 30 S

A working app, and what we learned

Key message: The app, conclusions and next steps



Finally, the working app and what we learned. *[Point to the left screenshot.]* On the left, the worn-sign YOLO finds five objects in a street photo. *[Point to the right screenshot.]* On the right, the app shows the fuzzy stages and both answers for a green light, at 96% confidence. The app runs every step live, and it can be opened on a phone.

We reached three conclusions. **First**, training on damaged images beats repairing them: 0.819 against 0.715 macro F1. **Second**, fuzzy enhancement still helps every model that never saw damage, and it leaves clean images alone. **Third**, real photos showed weaknesses that no test set covered: too few classes, close-up lights and worn paint. We fixed each one and measured it before keeping it.

There are limitations too. A 32 by 32 input cannot read text or numbers on a sign, and extremely faded signs still fail. So, as next steps, we plan a larger input or a pretrained backbone to read text and numbers, stronger sun-bleaching damage with real photos of old signs, and night and rain photo tests, with YOLO trained on damaged frames.

Thank you for listening. I am happy to answer your questions.